CERTIK

RWAlpha

Preliminary Comments

CertiK Assessed on Jul 10th, 2026



CertiK Assessed on Jul 10th, 2026

RWAlpha

These preliminary comments were prepared by CertiK.

Executive Summary

TYPES	ECOSYSTEM	METHODS
ERC-20	Binance Smart Chain (BSC)	Formal Verification, Manual Review, Static Analysis

LANGUAGE	TIMELINE
Solidity	Preliminary comments published on 04/15/2026

Vulnerability Summary



15	1	1	11	0	2
Total Findings	Resolved	Partially Resolved	Acknowledged	Declined	Pending

1	Centralization	1 Acknowledged	Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.
0	Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
0	Major		Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.
3	Medium	3 Acknowledged	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
4	Minor	1 Partially Resolved, 2 Acknowledged, 1 Pending	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
7	Informational	1 Resolved, 5 Acknowledged, 1 Pending	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.
0	Discussion		The impact of the issue is yet to be determined, hence requires further clarifications from the project team.

TABLE OF CONTENTS | RWALPHA

Audit Summary

Executive Summary

Vulnerability Summary

Codebase

Audit Scope

Approach & Methods

Review Notes

Overview

External Dependencies

Privileged Functions

Findings

DMA-01 : Centralization Related Risks

DMA-02 : Inconsistent Access Control Allows Operator To Update NAV

DMA-03 : NAV Not Enforced In Mint And Burn Settlement

DMA-04 : Mint Refunds And Burn Payouts Rely On External Wallets

DMA-05 : Pending Requests Depend On Operator Action

DMA-06 : Statement Date Validation Is Insufficient

DMA-07 : Missing Minimum Request Size Can Increase Operational Overhead

RWA-01 : Minimum Mint Amount Is Fully Controlled By Operator

DMA-08 : Missing Emit Events

DMA-09 : Hardcoded Share Decimals

DMA-10 : Daily Statement Is Not Validated Against On-Chain State

DMA-11 : Batch Processing Can Be Blocked By One Failed Request

DMA-12 : Settlement Assumes Standard Stablecoin Transfers

DMA-13 : Confirmation Of Project Design

RWA-02 : Zero-State Daily Statements Cannot Be Published

Formal Verification

Considered Functions And Scope

Verification Results

Appendix

Disclaimer

CODEBASE | RWALPHA

Repository

https://github.com/dmzFinance/rwalpha-contract

Commit

- 45ffe0e7bf5705da436ee9dab327a9871d132377
- b795757943fecd15d8e097652f2ca5f67e16fd69
- 47074874d116c021d431d03f1496bc24dfe6852b
- 06b245efd3c404c483342e5d71d005c650c66236

AUDIT SCOPE | RWALPHA

dmzFinance/rwalpha-contract
NonTransferableFundController.sol
FundController.sol
FundController.sol
FundController.sol

APPROACH & METHODS | RWALPHA

This audit was conducted for DMZ to evaluate the security and correctness of the smart contracts associated with the RWAlpha project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Static Analysis and Manual Review.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | RWALPHA

Overview

FundController is a Solidity-based fund share management contract. It extends `ERC20` and implements role-based workflows for mint requests, burn requests, NAV updates, and daily statement publication.

External Dependencies

The contract interacts with one or more third-party protocols and contracts to fulfill its business logic. The scope of the audit treats third-party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised and this may lead to lost or stolen assets. In addition, upgrades or behavior deviations of third parties can create severe impacts, including loss of funds or settlement failures.

The following external addresses are used within the `FundController` contract:

- `stablecoin`: Constructor-specified ERC20 token used for subscriptions, refunds, and payouts.
- `mintCollectionWallet`: Admin-configurable external wallet that receives stablecoins on mint requests.
- `burnPayoutWallet`: Admin-configurable external wallet that provides stablecoin payouts for burn settlements.
- `admin_`: Constructor-specified account granted `DEFAULT_ADMIN_ROLE`.
- `operator_`: Constructor-specified account granted `OPERATOR_ROLE`.

The following interfaces/libraries are used:

- `IERC20`: Interface for stablecoin interactions.
- `SafeERC20`: Library for safer ERC20 transfer operations.

Also, the following libraries/contracts are considered as third-party dependencies:

- `@openzeppelin/contracts/access/AccessControl.sol`
- `@openzeppelin/contracts/token/ERC20/ERC20.sol`
- `@openzeppelin/contracts/token/ERC20/IERC20.sol`
- `@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol`
- `@openzeppelin/contracts/utils/ReentrancyGuard.sol`

We assume these contracts and external addresses are valid and non-malicious actors implementing correct logic to collaborate with the current project.

Privileged Functions

In the **FundController** contract, privileged roles are adopted to ensure the dynamic runtime updates of the project, which are specified in the **Centralization Risks** findings below.

The advantage of those privileged roles in the codebase is that the client reserves the ability to adjust the protocol according to the runtime requirements to best serve the community. It is also worth noting the potential drawbacks of these functions,

which should be clearly stated through the client's action/plan. Additionally, if the private keys of the privileged accounts are compromised, it could lead to devastating consequences for the project.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community. Any plan to invoke the aforementioned functions should be also considered to move to the execution queue of a `Timelock` contract.

FINDINGS | RWALPHA



15	0	1	0	3	4	7	0
Total Findings	Critical	Centralization	Major	Medium	Minor	Informational	Discussion

This report has been prepared for DMZ to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 15 issues were identified. Leveraging a combination of Static Analysis & Manual Review the following findings were uncovered:

ID	Title	Category	Severity	Status
DMA-01	**Centralization Related Risks**	**Centralization**	**Centralization**	**Acknowledged**
DMA-02	Inconsistent Access Control Allows Operator To Update NAV	Inconsistency	Medium	Acknowledged
DMA-03	NAV Not Enforced In Mint And Burn Settlement	Volatile Code	Medium	Acknowledged
DMA-04	Mint Refunds And Burn Payouts Rely On External Wallets	Logical Issue	Medium	Acknowledged
DMA-05	Pending Requests Depend On Operator Action	Logical Issue, Design Issue	Minor	Acknowledged
DMA-06	Statement Date Validation Is Insufficient	Logical Issue	Minor	Acknowledged
DMA-07	Missing Minimum Request Size Can Increase Operational Overhead	Design Issue	Minor	Partially Resolved
RWA-01	Minimum Mint Amount Is Fully Controlled By Operator	Logical Issue	Minor	Pending
DMA-08	Missing Emit Events	Coding Style	Informational	Acknowledged
DMA-09	Hardcoded Share Decimals	Coding Issue	Informational	Resolved
DMA-10	Daily Statement Is Not Validated Against On-Chain State	Inconsistency, Logical Issue	Informational	Acknowledged

ID	Title	Category	Severity	Status
DMA-11	Batch Processing Can Be Blocked By One Failed Request	Coding Issue	Informational	Acknowledged
DMA-12	Settlement Assumes Standard Stablecoin Transfers	Logical Issue	Informational	Acknowledged
DMA-13	Confirmation Of Project Design	Design Issue	Informational	Acknowledged
RWA-02	Zero-State Daily Statements Cannot Be Published	Volatile Code	Informational	Pending

DMA-01 | Centralization Related Risks

Category	Severity	Location	Status
Centralization	Centralization	NonTransferableFundController.sol: 170, 174, 235, 243, 250, 258, 265, 278, 290, 303, 315, 327, 347, 365	Acknowledged

Description

`NonTransferableFundController` is used in an RWAlpha workflow that follows an “on-chain request + off-chain operational settlement” model. In this business context, the workflow relies on centralized operators to perform NAV updates, mint/redemption review, custody reconciliation, and final settlement actions. The protocol design intentionally depends on operational roles and wallets.

In the contract `NonTransferableFundController`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

- `setNav(uint256 newNav)`
- `setMintCollectionWallet(address newMintCollectionWallet)`
- `setBurnPayoutWallet(address newBurnPayoutWallet)`
- `grantRole(bytes32 role, address account)`
- `revokeRole(bytes32 role, address account)`

If a `DEFAULT_ADMIN_ROLE` account is compromised, the malicious actor can take control of core governance and cash-flow controls of the RWAlpha system: alter NAV inputs used for share pricing, reroute mint collection and redemption payout wallets, and reassign privileged roles. In an RWA context, this can break the integrity of issuance/redemption processing, disrupt custody-to-ledger reconciliation, and causing unfair settlements, blocked redemptions, and material financial loss.

The role `OPERATOR_ROLE` has authority over the following functions:

- `function setMinMintStableAmount(uint256 newMinAmount)`
- `settleMint(uint256 requestId, uint256 mintedAmount, bytes32 memo)`
- `rejectMint(uint256 requestId, bytes32 memo)`
- `settleBurn(uint256 requestId, uint256 payoutAmount, bytes32 memo)`
- `rejectBurn(uint256 requestId, bytes32 memo)`
- `batchSettleMint(uint256[] calldata requestIds, uint256[] calldata mintedAmounts, bytes32[] calldata memos)`
- `batchRejectMint(uint256[] calldata requestIds, bytes32[] calldata memos)`
- `batchSettleBurn(uint256[] calldata requestIds, uint256[] calldata payoutAmounts, bytes32[] calldata memos)`
- `batchRejectBurn(uint256[] calldata requestIds, bytes32[] calldata memos)`

- `publishDailyStatement(uint32 statementDate, uint256 statementNav, uint256 totalShares, uint256 netAssets, bytes32 fileHash, string calldata uri)`
- `publishDailyStatementAndSetNav(uint32 statementDate, uint256 statementNav, uint256 totalShares, uint256 netAssets, bytes32 fileHash, string calldata uri)`

If an `OPERATOR_ROLE` account is compromised, the malicious actor can directly control the operational execution layer of the RWAlpha lifecycle: they can decide which mint/burn requests are processed or rejected, set discretionary settlement amounts, and publish statement/NAV data that downstream systems may treat as authoritative. In an RWA setting, this can undermine issuance and redemption fairness, corrupt investor-level entitlement records, break accounting and custody reconciliation, and ultimately lead to financial loss.

In addition, `NonTransferableFundController` contract relies on two protocol-controlled addresses: **`mintCollectionWallet`** receives USDT when users mint, and **`burnPayoutWallet`** pays USDT on redemption via `transferFrom`, so that wallet must **approve** the contract with sufficient USDT allowance or redemptions fail. Both addresses are centralized operational hot spots. They must be protected; **multisig (e.g. Gnosis Safe)** is strongly recommended, together with key hygiene, allowance policy, and monitoring. Loss or compromise of either wallet can block exits or misappropriate user funds.

Recommendation

The identified risk reflects the current RWAlpha operating model of this project.
At this stage, privileged roles are business-critical for NAV updates, mint/redemption operations, statement publishing, and wallet administration, so these functions should not be removed directly.

The practical objective is therefore to reduce key-person and key-compromise risk through stronger governance and operational controls, while preserving required business functionality.

We recommend the team prioritize secure custody and governance of privileged accounts, and progressively harden the control model (e.g., multisig-based administration, timelock, role separation, and transparent governance disclosures).

Short Term

Use operational hardening to reduce immediate compromise risk.

- Assign `DEFAULT_ADMIN_ROLE` and `OPERATOR_ROLE` to multisig wallets (e.g., 2/3 or 3/5), not single EOAs.
- Introduce timelock (e.g., ~48 hours) for high-impact admin actions such as role changes and wallet updates.

Long Term

Increase governance robustness and reduce trust concentration over time.

- Separate duties across distinct roles/multisigs (e.g., NAV management, settlement execution, role administration, fund wallet control).
- Add governance/oversight mechanisms (committee or DAO-based approval layer) for sensitive parameter and role changes.

Alleviation

[DMZ, 04/15/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

[CertiK, 04/15/2026]: It is suggested to implement the aforementioned methods to avoid centralized failure. Also, CertiK strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles. The finding will be updated when further information is provided.

DMA-02 | Inconsistent Access Control Allows Operator To Update NAV

Category	Severity	Location	Status
Inconsistency	Medium	FundController.sol (04/03-45ffe0): 161~163, 346~347	Acknowledged

Description

The `setNav()` function is used to update NAV directly and is restricted to `DEFAULT_ADMIN_ROLE`. The `publishDailyStatementAndSetNav()` function is used to publish a daily statement and update NAV in the same transaction.

However, these two functions do not enforce the same permission model. While `setNav()` is admin-only, `publishDailyStatementAndSetNav()` is only restricted by `OPERATOR_ROLE`, and it directly calls `_setNav(statementNav)`.

```
161 function setNav(uint256 newNav) external onlyRole(DEFAULT_ADMIN_ROLE) {
162     _setNav(newNav);
163 }
```

```
339 function publishDailyStatementAndSetNav(...) external onlyRole(OPERATOR_ROLE) {
340     _setNav(statementNav);
341     _publishDailyStatement(...);
342 }
```

As a result, an operator can still update NAV through `publishDailyStatementAndSetNav()` even though direct NAV updates appear to be restricted to admins. This creates inconsistent access control and may allow operators to modify a critical pricing parameter with more authority than intended, which can lead to unintended NAV changes and inconsistent fund accounting.

Recommendation

Use the same access control for all NAV update functions. If NAV updates should be admin-only, `publishDailyStatementAndSetNav()` should not allow operators to call `_setNav()`.

Alleviation

[DMZ, 04/15/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

DMA-03 | NAV Not Enforced In Mint And Burn Settlement

Category	Severity	Location	Status
Volatile Code	Medium	FundController.sol (04/03-45ffe0): 218	Acknowledged

Description

`requestMint()` and `requestBurn()` are used to create fund subscription and redemption requests, while `settleMint()` and `settleBurn()` are used by the operator to finalize those requests by determining how many shares are minted or how much stablecoin is paid out.

However, although the contract stores a `nav` value and provides functions to update it, the settlement logic does not use `nav` at all. Instead, `settleMint()` takes `mintedAmount` directly from the operator, and `settleBurn()` takes `payoutAmount` directly from the operator.

As a result, the contract does not enforce any NAV-based relationship between deposited stablecoins, minted shares, burned shares, and redemption payouts. The final settlement amounts are fully determined by operator input, which may lead to inconsistent pricing and incorrect fund accounting.

```
213 function settleMint(
214     uint256 requestId,
215     uint256 mintedAmount,
216     bytes32 memo
217 ) external onlyRole(OPERATOR_ROLE) nonReentrant {
218     _settleMint(requestId, mintedAmount, memo);
219 }
```

```
228 function settleBurn(
229     uint256 requestId,
230     uint256 payoutAmount,
231     bytes32 memo
232 ) external onlyRole(OPERATOR_ROLE) nonReentrant {
233     _settleBurn(requestId, payoutAmount, memo);
234 }
```

Recommendation

Use `nav` to determine mint and burn settlement amounts.

Alleviation

[DMZ, 04/15/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

DMA-04 | Mint Refunds And Burn Payouts Rely On External Wallets

Category	Severity	Location	Status
Logical Issue	Medium	FundController.sol (04/03-45ffe0): 172~173, 441~442, 458	Acknowledged

Description

`requestMint()` and `rejectMint()` are used to create and reject mint requests, while `requestBurn()` and `settleBurn()` are used to create and settle burn requests.

However, these flows rely on external wallets to return or pay stablecoins. In the mint flow, `requestMint()` immediately transfers the user's stablecoins to `mintCollectionWallet`, and `_rejectMint()` later refunds the user by pulling funds back from that external wallet. In the burn flow, the user's shares are first transferred to the contract, but `_settleBurn()` pays the user from `burnPayoutWallet` rather than from contract-held funds.

As a result, both mint refunds and burn payouts depend on external wallets continuing to hold sufficient funds and maintain token approval for the contract, which may lead to failed refunds or failed redemptions.

```
172  stablecoin.safeTransferFrom(msg.sender, mintCollectionWallet, stableAmount);
```

```
441  stablecoin.safeTransferFrom(r.collectionWallet, r.user, r.stableAmount);
```

```
458  stablecoin.safeTransferFrom(r.payoutWallet, r.user, payoutAmount);
```

Recommendation

Keep stablecoins in the contract until mint or burn requests are finalized, and perform refunds and payouts directly from the contract balance instead of relying on external wallets.

Alleviation

[DMZ, 04/15/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

DMA-05 | Pending Requests Depend On Operator Action

Category	Severity	Location	Status
Logical Issue, Design Issue	Minor	FundController.sol (04/03-45ffe0): 165, 189, 213, 221, 228, 236	Acknowledged

Description

`requestMint()` and `requestBurn()` are used to create mint and burn requests, while `settleMint()`, `rejectMint()`, `settleBurn()`, and `rejectBurn()` are used to finalize those requests.

However, once a request is created, the contract does not provide any user-side timeout, cancellation, or recovery path. A pending request can only be resolved through operator action. As a result, if the operator delays processing or becomes unavailable, user funds or shares may remain locked until the operator processes the request, which can result in fund or share lockup and reduced user exit certainty.

Recommendation

Add a user-side timeout or cancellation mechanism for pending requests.

Alleviation

[DMZ, 04/15/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

DMA-06 | Statement Date Validation Is Insufficient

Category	Severity	Location	Status
Logical Issue	Minor	FundController.sol (04/03-45ffe0): 389	Acknowledged

Description

`publishDailyStatement()` and `publishDailyStatementAndSetNav()` are used to publish daily statements for a given `statementDate`.

However, the contract only checks that `statementDate` is not zero, and does not validate whether it represents a meaningful or expected statement date. For example, it does not verify date format, correctness, or whether the date is reasonable relative to the current reporting period.

As a result, the contract may accept daily statements with invalid or inconsistent date values, which can lead to reporting errors and reduce the reliability of statement records.

Recommendation

Add validation to ensure `statementDate` follows the intended date format and reporting rules.

Alleviation

[DMZ, 04/15/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

DMA-07 | Missing Minimum Request Size Can Increase Operational Overhead

Category	Severity	Location	Status
Design Issue	Minor	FundController.sol (04/03-45ffe0): 170, 194	Partially Resolved

Description

The `FundController` contract accepts any non-zero amount in both `requestMint` and `requestBurn`. As a result, users can submit dust-sized mint or burn requests that create `Pending` request records with negligible business value. Although this behavior does not directly cause an on-chain denial of service, because requests are tracked by ID and operators can process requests in arbitrary order, it can still increase operational overhead. In particular, a large number of dust requests may add noise to off-chain review workflows, increase batch processing frequency, and expand historical on-chain state volume.

Impact

- Increased operational overhead for operators reviewing and processing low-value requests.
- Larger on-chain request history and event footprint.
- Potential degradation of off-chain workflow efficiency if many economically insignificant requests are submitted.

Recommendation

Consider introducing configurable minimum thresholds such as `minMintAmount` and `minBurnAmount`.

Alleviation

[DMZ, 04/15/2026]:

Issue acknowledged. Changes have been reflected in the commit hash: https://github.com/dmzFinance/rwalpha-contract/commit/b795757943fecd15d8e097652f2ca5f67e16fd69

[CertiK, 04/16/2026]:

The team introduced a maximum mint amount check in `requestMint()` in commit b795757943fecd15d8e097652f2ca5f67e16fd69.

However, this is not the expected remediation for the reported issue. The finding concerns the lack of a minimum mint or burn threshold, and the contract still does not enforce any minimum request size. As a result, dust-sized requests remain possible.

[DMZ, 04/16/2026]:

Issue acknowledged. Changes have been reflected in the commit hash: https://github.com/dmzFinance/rwalpha-contract/commit/47074874d116c021d431d03f1496bc24dfe6852b

[CertiK, 04/16/2026]:

The team partially resolved the finding by introducing a minimum mint amount check in `requestMint()` and changes were reflected in commit 47074874d116c021d431d03f1496bc24dfe6852b.

It's noted that the contract does not enforce minimum burn threshold, so dust-sized requests remain possible.

RWA-01 | Minimum Mint Amount Is Fully Controlled By Operator

Category	Severity	Location	Status
Logical Issue	Minor	NonTransferableFundController.sol (07/07-06b245): 178~179	Pending

Description

`setMinMintStableAmount()` is used to update the minimum stablecoin amount required for users to create mint requests.

However, this function is controlled by `OPERATOR_ROLE` and only enforces a lower bound. It does not enforce any upper bound on the new minimum mint amount.

```
178  uint256 minimumAllowed = 100 * (10 ** _tokenDecimals);
179  if (newMinAmount < minimumAllowed) revert InvalidAmount();
```

As a result, an operator can set the minimum mint amount to an extremely high value. This can effectively prevent most users from creating new mint requests, even though the mint flow itself remains enabled.

Recommendation

Add a reasonable upper bound for `newMinAmount`, or restrict this parameter update to admin/governance control.

DMA-08 | Missing Emit Events

Category	Severity	Location	Status
Coding Style	Informational	FundController.sol (04/03-45ffe0): 151, 152, 153, 154	Acknowledged

Description

The contract performs one or more critical state initializations without dedicated protocol-level events. This reduces transparency for off-chain indexers and auditors that rely on explicit business events to monitor governance-sensitive configuration.

The following marked locations initialize privileged configuration without dedicated custom events:

- **FundController**:

```
151         stablecoin = IERC20(stablecoin_);
```

```
152         mintCollectionWallet = mintCollectionWallet_;
```

```
153         burnPayoutWallet = burnPayoutWallet_;
```

```
154         nav = initialNav_;
```

While post-deployment updates emit dedicated events, these initial privileged values are set only during construction without dedicated protocol events describing the initialized parameters in a single business-level log.

Recommendation

It is suggested to declare and emit corresponding events for all the essential state variables that are possible to be changed during runtime.

Alleviation

[DMZ, 04/15/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

DMA-09 | Hardcoded Share Decimals

Category	Severity	Location	Status
Coding Issue	Informational	FundController.sol (04/03-45ffe0): 158	Resolved

Description

The `decimals()` function is used to define the precision of the share token. In the current implementation, it always returns `6`, which fixes the share token to 6-decimal precision regardless of the stablecoin configuration or any external accounting assumptions.

While this may be intentional, a fixed 6-decimal model can still create integration or accounting inconsistencies if off-chain NAV calculations, settlement systems, or future asset integrations use a different unit convention. The implementation should ensure that this precision choice is intentional and consistently applied across the full mint, burn, and accounting workflow.

Recommendation

Ensure the 6-decimal share model is intentional and consistently used across all off-chain accounting, NAV calculations, and integrations.

Alleviation

[DMZ, 04/15/2026]: The team heeded the advice and resolved the issue by removing the hardcoded 6-decimal setting and making `decimals()` return the configured stablecoin's decimals through `_tokenDecimals` in commit b795757943fecd15d8e097652f2ca5f67e16fd69.

DMA-10 | Daily Statement Is Not Validated Against On-Chain State

Category	Severity	Location	Status
Inconsistency, Logical Issue	Informational	FundController.sol (04/03-45ffe0): 389~392	Acknowledged

Description

`publishDailyStatement()` is used to publish a daily statement, and `publishDailyStatementAndSetNav()` is used to publish a daily statement while updating NAV in the same transaction. These functions record values such as `statementNav`, `totalShares`, and `netAssets` on-chain.

However, the contract does not validate whether these reported values match the actual on-chain state. In `_publishDailyStatement()`, the inputs are only checked for basic validity such as nonzero values and duplicate dates, but they are not compared against values such as `totalSupply()` or any contract-held assets.

As a result, the contract can publish daily statements that do not reflect the real on-chain state, which may lead to inaccurate reporting and inconsistent fund accounting.

Recommendation

Validate statement fields against the relevant on-chain state before publishing.

Alleviation

[DMZ, 04/15/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

DMA-11 | Batch Processing Can Be Blocked By One Failed Request

Category	Severity	Location	Status
Coding Issue	Informational	FundController.sol (04/03-45ffe0): 252, 264, 277, 289	Acknowledged

Description

`batchSettleMint()`, `batchRejectMint()`, `batchSettleBurn()`, and `batchRejectBurn()` are used to process multiple requests in a single transaction.

However, each function processes requests in a loop without handling individual failures. As a result, if any single request reverts due to an invalid status, invalid amount, or failed external token transfer, the entire batch reverts and none of the other requests can be processed. This may allow one failed request to block the processing of other valid requests in the same batch and lead to delays in processing other valid requests.

In addition, the batch handlers impose no upper bound on `length`, unbounded growth risks out-of-gas (OOG) before completion.

Recommendation

Handle individual request failures separately and enforce a conservative `MAX_BATCH` on every batch entry point.

Alleviation

[DMZ, 04/15/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

DMA-12 | Settlement Assumes Standard Stablecoin Transfers

Category	Severity	Location	Status
Logical Issue	Informational	FundController.sol (04/03-45ffe0): 172, 441, 458~459	Acknowledged

Description

`requestMint()`, `rejectMint()`, and `settleBurn()` are used to move stablecoins during mint creation, mint rejection, and burn settlement.

However, these flows assume that the transferred amount always matches the requested amount. The contract records the user-provided `stableAmount` in `requestMint()`, refunds the same amount in `_rejectMint()`, and pays the operator-provided `payoutAmount` in `_settleBurn()`, but it does not verify the actual amount received or sent during token transfers.

As a result, if the configured stablecoin has non-standard transfer behavior, such as transfer fees or other balance-changing mechanics, the recorded request amounts may not match the actual token amounts moved. This can lead to accounting inconsistencies, failed refunds, or failed burn payouts.

Recommendation

Use a stablecoin with standard ERC20 transfer behavior, or validate the actual amount received and paid during settlement.

Alleviation

[DMZ, 04/15/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

DMA-13 | Confirmation Of Project Design

Category	Severity	Location	Status
Design Issue	Informational	FundController.sol (04/03-45ffe0): 416, 446	Acknowledged

Description

Based on the project documentation, it is positioned for an RWA use case with off-chain operations and settlement. The current design appears intentionally simple, we have some concerns regarding on the following items:

First, the contract does not introduce explicit compliance gating at the smart-contract layer (for example, no on-chain KYC gating, whitelist restrictions).

Second, the fund share token is implemented as a transferable ERC20 and can be freely transferred between addresses. Please confirm whether unrestricted peer-to-peer transfer is aligned with the intended RWA compliance model, especially given that the design documentation indicates interest distribution / yield-related operations.

In addition, settlement amounts (`mintedAmount` and `payoutAmount`) are still operator-provided with only `> 0` checks in settlement paths. We would like to confirm the off-chain pricing formula, fee policy, investor entitlement logic (including interest distribution), and reconciliation controls between off-chain books and on-chain execution.

Recommendation

Ensure the intended compliance and settlement model is clearly documented and consistently enforced across off-chain operations and on-chain behavior. In particular, confirm whether unrestricted ERC20 transfers are compatible with the target investor eligibility model, and document the off-chain controls governing pricing, settlement, entitlement calculation, and reconciliation.

Alleviation

[DMZ, 04/15/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

[CertiK, 07/10/2026]: The team have made changes to make the the funds can NOT be freely transferred between addresses.

RWA-02 | Zero-State Daily Statements Cannot Be Published

Category	Severity	Location	Status
Volatile Code	Informational	NonTransferableFundController.sol (07/07-06b245): 428	Pending

Description

`publishDailyStatement()` and `publishDailyStatementAndSetNav()` are used by the operator to publish daily fund statements, including NAV, total shares, net assets, file hash, and statement URI.

However, the shared `_publishDailyStatement()` logic requires `statementNav`, `totalShares`, and `netAssets` to all be non-zero:

```
428  if (statementNav == 0 || totalShares == 0 || netAssets == 0) revert
InvalidAmount();
```

As a result, the contract cannot publish a daily statement when the fund legitimately reaches a zero state. This can create gaps in the on-chain statement history and make the reporting record inconsistent with the fund's actual lifecycle.

Recommendation

Allow zero `totalShares` or `netAssets` when they are valid under the fund lifecycle, or add explicit handling for empty or wind-down fund states.

FORMAL VERIFICATION | RWALPHA

Formal guarantees about the behavior of smart contracts can be obtained by reasoning about properties relating to the entire contract (e.g. contract invariants) or to specific functions of the contract. Once such properties are proven to be valid, they guarantee that the contract behaves as specified by the property. As part of this audit, we applied formal verification to prove that important functions in the smart contracts adhere to their expected behaviors.

Considered Functions And Scope

Considered Functions And Scope

Verification of ERC-20 Compliance

We verified properties of the public interface of those token contracts that implement the ERC-20 interface. This covers

- Functions `transfer` and `transferFrom` that are widely used for token transfers,
- functions `approve` and `allowance` that enable the owner of an account to delegate a certain subset of her tokens to another account (i.e. to grant an allowance), and
- the functions `balanceOf` and `totalSupply`, which are verified to correctly reflect the internal state of the contract.

The properties that were considered within the scope of this audit are as follows:

Property Name	Title
erc20-transfer-revert-zero	`transfer` Prevents Transfers to the Zero Address
erc20-approve-correct-amount	`approve` Updates the Approval Mapping Correctly
erc20-transfer-exceed-balance	`transfer` Fails if Requested Amount Exceeds Available Balance
erc20-transferfrom-fail-exceed-allowance	`transferFrom` Fails if the Requested Amount Exceeds the Available Allowance
erc20-transfer-correct-amount	`transfer` Transfers the Correct Amount in Transfers
erc20-balanceof-change-state	`balanceOf` Does Not Change the Contract's State
erc20-allowance-change-state	`allowance` Does Not Change the Contract's State
erc20-totalsupply-change-state	`totalSupply` Does Not Change the Contract's State
erc20-transferfrom-fail-exceed-balance	`transferFrom` Fails if the Requested Amount Exceeds the Available Balance
erc20-transferfrom-correct-amount	`transferFrom` Transfers the Correct Amount in Transfers

Property Name	Title
erc20-transferfrom-correct-allowance	`transferFrom` Updated the Allowance Correctly
erc20-transferfrom-fail-recipient-overflow	`transferFrom` Prevents Overflows in the Recipient's Balance
erc20-transfer-recipient-overflow	`transfer` Prevents Overflows in the Recipient's Balance
erc20-approve-false	If `approve` Returns `false`, the Contract's State Is Unchanged
erc20-approve-never-return-false	`approve` Never Returns `false`
erc20-totalsupply-correct-value	`totalSupply` Returns the Value of the Corresponding State Variable
erc20-allowance-succeed-always	`allowance` Always Succeeds
erc20-transferfrom-false	If `transferFrom` Returns `false`, the Contract's State Is Unchanged
erc20-approve-revert-zero	`approve` Prevents Approvals For the Zero Address
erc20-approve-succeed-normal	`approve` Succeeds for Valid Inputs
erc20-totalsupply-succeed-always	`totalSupply` Always Succeeds
erc20-transferfrom-never-return-false	`transferFrom` Never Returns `false`
erc20-balanceof-correct-value	`balanceOf` Returns the Correct Value
erc20-transferfrom-revert-zero-argument	`transferFrom` Fails for Transfers with Zero Address Arguments
erc20-transfer-false	If `transfer` Returns `false`, the Contract State Is Not Changed
erc20-allowance-correct-value	`allowance` Returns Correct Value
erc20-transfer-never-return-false	`transfer` Never Returns `false`
erc20-balanceof-succeed-always	`balanceOf` Always Succeeds

Verification Results

In the remainder of this section, we list all contracts where formal verification of at least one property was not successful. There are several reasons why this could happen:

- False: The property is violated by the project.
- Inconclusive: The proof engine cannot prove or disprove the property due to timeouts or exceptions.
- Inapplicable: The property does not apply to the project.

Detailed Results For Contract FundController (FundController.sol) In Commit

47074874d116c021d431d03f1496bc24dfe6852b

Verification of ERC-20 Compliance

Detailed Results for Function `transfer`

Property Name	Final Result	Remarks
erc20-transfer-revert-zero	True	
erc20-transfer-exceed-balance	True	
erc20-transfer-correct-amount	True	
erc20-transfer-recipient-overflow	Inconclusive	
erc20-transfer-false	True	
erc20-transfer-never-return-false	True	

Detailed Results for Function `approve`

Property Name	Final Result	Remarks
erc20-approve-correct-amount	True	
erc20-approve-false	True	
erc20-approve-never-return-false	True	
erc20-approve-revert-zero	True	
erc20-approve-succeed-normal	True	

Detailed Results for Function `transferFrom`

Property Name	Final Result	Remarks
erc20-transferfrom-fail-exceed-allowance	True	
erc20-transferfrom-fail-exceed-balance	True	
erc20-transferfrom-correct-amount	True	
erc20-transferfrom-correct-allowance	True	
erc20-transferfrom-fail-recipient-overflow	Inconclusive	
erc20-transferfrom-false	True	
erc20-transferfrom-never-return-false	True	
erc20-transferfrom-revert-zero-argument	True	

Detailed Results for Function `balanceOf`

Property Name	Final Result	Remarks
erc20-balanceof-change-state	True	
erc20-balanceof-correct-value	True	
erc20-balanceof-succeed-always	True	

Detailed Results for Function `allowance`

Property Name	Final Result	Remarks
erc20-allowance-change-state	True	
erc20-allowance-succeed-always	True	
erc20-allowance-correct-value	True	

Detailed Results for Function `totalSupply`

Property Name	Final Result	Remarks
erc20-totalsupply-change-state	True	
erc20-totalsupply-correct-value	True	
erc20-totalsupply-succeed-always	True	

Detailed Results For Contract FundController (FundController.sol) In Commit b795757943fecd15d8e097652f2ca5f67e16fd69

Verification of ERC-20 Compliance

Detailed Results for Function `balanceOf`

Property Name	Final Result	Remarks
erc20-balanceof-succeed-always	True	
erc20-balanceof-correct-value	True	
erc20-balanceof-change-state	True	

Detailed Results for Function `transferFrom`

Property Name	Final Result	Remarks
erc20-transferfrom-revert-zero-argument	True	
erc20-transferfrom-false	True	
erc20-transferfrom-fail-exceed-allowance	True	
erc20-transferfrom-correct-allowance	True	
erc20-transferfrom-fail-exceed-balance	True	
erc20-transferfrom-correct-amount	True	
erc20-transferfrom-fail-recipient-overflow	Inconclusive	
erc20-transferfrom-never-return-false	True	

Detailed Results for Function `transfer`

Property Name	Final Result	Remarks
erc20-transfer-exceed-balance	True	
erc20-transfer-revert-zero	True	
erc20-transfer-false	True	
erc20-transfer-never-return-false	True	
erc20-transfer-correct-amount	True	
erc20-transfer-recipient-overflow	Inconclusive	

Detailed Results for Function `approve`

Property Name	Final Result	Remarks
erc20-approve-correct-amount	True	
erc20-approve-never-return-false	True	
erc20-approve-false	True	
erc20-approve-revert-zero	True	
erc20-approve-succeed-normal	True	

Detailed Results for Function `allowance`

Property Name	Final Result	Remarks
erc20-allowance-change-state	True	
erc20-allowance-correct-value	True	
erc20-allowance-succeed-always	True	

Detailed Results for Function `totalSupply`

Property Name	Final Result	Remarks
erc20-totalsupply-change-state	True	
erc20-totalsupply-correct-value	True	
erc20-totalsupply-succeed-always	True	

Detailed Results For Contract FundController (FundController.sol) In Commit 45ffe0e7bf5705da436ee9dab327a9871d132377

Verification of ERC-20 Compliance

Detailed Results for Function `transferFrom`

Property Name	Final Result	Remarks
erc20-transferfrom-never-return-false	True	
erc20-transferfrom-false	True	
erc20-transferfrom-revert-zero-argument	True	
erc20-transferfrom-fail-exceed-allowance	True	
erc20-transferfrom-fail-exceed-balance	True	
erc20-transferfrom-correct-allowance	True	
erc20-transferfrom-correct-amount	True	
erc20-transferfrom-fail-recipient-overflow	Inconclusive	

Detailed Results for Function `totalSupply`

Property Name	Final Result	Remarks
erc20-totalsupply-correct-value	True	
erc20-totalsupply-change-state	True	
erc20-totalsupply-succeed-always	True	

Detailed Results for Function `transfer`

Property Name	Final Result	Remarks
erc20-transfer-never-return-false	True	
erc20-transfer-false	True	
erc20-transfer-revert-zero	True	
erc20-transfer-exceed-balance	True	
erc20-transfer-correct-amount	True	
erc20-transfer-recipient-overflow	Inconclusive	

Detailed Results for Function `balanceOf`

Property Name	Final Result	Remarks
erc20-balanceof-correct-value	True	
erc20-balanceof-change-state	True	
erc20-balanceof-succeed-always	True	

Detailed Results for Function `allowance`

Property Name	Final Result	Remarks
erc20-allowance-correct-value	True	
erc20-allowance-change-state	True	
erc20-allowance-succeed-always	True	

Detailed Results for Function `approve`

Property Name	Final Result	Remarks
erc20-approve-correct-amount	True	
erc20-approve-never-return-false	True	
erc20-approve-false	True	
erc20-approve-succeed-normal	True	
erc20-approve-revert-zero	True	

APPENDIX | RWALPHA

Finding Categories

Categories	Description
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.

Details on Formal Verification

Some Solidity smart contracts from this project have been formally verified. Each such contract was compiled into a mathematical model that reflects all its possible behaviors with respect to the property. The model takes into account the semantics of the Solidity instructions found in the contract. All verification results that we report are based on that model.

The following assumptions and simplifications apply to our model:

- Certain low-level calls and inline assembly are not supported and may lead to a contract not being formally verified.
- We model the semantics of the Solidity source code and not the semantics of the EVM bytecode in a compiled contract.

Formalism for property specifications

All properties are expressed in a behavioral interface specification language that CertiK has developed for Solidity, which allows us to specify the behavior of each function in terms of the contract state and its parameters and return values, as well as contract properties that are maintained by every observable state transition. Observable state transitions occur when the

contract's external interface is invoked and the invocation does not revert, and when the contract's Ether balance is changed by the EVM due to another contract's "self-destruct" invocation. The specification language has the usual Boolean connectives, as well as the operator `\old` (used to denote the state of a variable before a state transition), and several types of specification clause:

Apart from the Boolean connectives and the modal operators "always" (written `[]`) and "eventually" (written `<>`), we use the following predicates to reason about the validity of atomic propositions. They are evaluated on the contract's state whenever a discrete time step occurs:

- `requires [cond]` - the condition `cond`, which refers to a function's parameters, return values, and contract state variables, must hold when a function is invoked in order for it to exhibit a specified behavior.
- `ensures [cond]` - the condition `cond`, which refers to a function's parameters, return values, and both `\old` and current contract state variables, is guaranteed to hold when a function returns if the corresponding requires condition held when it was invoked.
- `invariant [cond]` - the condition `cond`, which refers only to contract state variables, is guaranteed to hold at every observable contract state.
- `constraint [cond]` - the condition `cond`, which refers to both `\old` and current contract state variables, is guaranteed to hold at every observable contract state except for the initial state after construction (because there is no previous state); constraints are used to restrict how contract state can change over time.

Description of the Analyzed ERC-20 Properties

Properties related to function `transfer`

erc20-transfer-correct-amount

All non-reverting invocations of `transfer(recipient, amount)` that return `true` must subtract the value in `amount` from the balance of `msg.sender` and add the same value to the balance of the `recipient` address.

Specification:

```
requires recipient != msg.sender;
requires balanceOf(recipient) + amount <= type(uint256).max;
ensures \result ==> balanceOf(recipient) == \old(balanceOf(recipient) + amount)
&& balanceOf(msg.sender) == \old(balanceOf(msg.sender) - amount);
  also
requires recipient == msg.sender;
ensures \result ==> balanceOf(msg.sender) == \old(balanceOf(msg.sender));
```

erc20-transfer-exceed-balance

Any transfer of an amount of tokens that exceeds the balance of `msg.sender` must fail.

Specification:

```
requires amount > balanceOf(msg.sender);
ensures !\result;
```

erc20-transfer-false

If the `transfer` function in contract `FundController` fails by returning `false`, it must undo all state changes it incurred before returning to the caller.

Specification:

```
ensures !\result ==> \assigned (\nothing);
```

erc20-transfer-never-return-false

The transfer function must never return `false` to signal a failure.

Specification:

```
ensures \result;
```

erc20-transfer-recipient-overflow

Any invocation of `transfer(recipient, amount)` must fail if it causes the balance of the `recipient` address to overflow.

Specification:

```
requires recipient != msg.sender;
requires balanceOf(recipient) + amount > type(uint256).max;
ensures !\result;
```

erc20-transfer-revert-zero

Any call of the form `transfer(recipient, amount)` must fail if the recipient address is the zero address.

Specification:

```
ensures \old(recipient) == address(0) ==> !\result;
```

Properties related to function `approve`

erc20-approve-correct-amount

All non-reverting calls of the form `approve(spender, amount)` that return `true` must correctly update the allowance mapping according to the address `msg.sender` and the values of `spender` and `amount`.

Specification:

```
requires spender != address(0);
ensures \result ==> allowance(msg.sender, \old(spender)) == \old(amount);
```

erc20-approve-false

If function `approve` returns `false` to signal a failure, it must undo all state changes that it incurred before returning to the caller.

Specification:

```
ensures !\result ==> \assigned (\nothing);
```

erc20-approve-never-return-false

The function `approve` must never returns `false`.

Specification:

```
ensures \result;
```

erc20-approve-revert-zero

All calls of the form `approve(spender, amount)` must fail if the address in `spender` is the zero address.

Specification:

```
ensures \old(spender) == address(0) ==> !\result;
```

erc20-approve-succeed-normal

All calls of the form `approve(spender, amount)` must succeed, if

- the address in `spender` is not the zero address and
- the execution does not run out of gas.

Specification:

```
requires spender != address(0);
ensures \result;
reverts_only_when false;
```

Properties related to function `transferFrom`

erc20-transferfrom-correct-allowance

All non-reverting invocations of `transferFrom(from, dest, amount)` that return `true` must decrease the allowance for address `msg.sender` over address `from` by the value in `amount`.

Specification:

```
ensures \result ==> allowance(\old(sender), msg.sender) == \old(allowance(sender,
msg.sender)) - \old(amount)
                  || (allowance(\old(sender), msg.sender) == \old(allowance(sender,
msg.sender)) && \old(allowance(sender, msg.sender)) == type(uint256).max);
```

erc20-transferfrom-correct-amount

All invocations of `transferFrom(from, dest, amount)` that succeed and that return `true` subtract the value in `amount` from the balance of address `from` and add the same value to the balance of address `dest`.

Specification:

```
requires recipient != sender;
requires balanceOf(recipient) + amount <= type(uint256).max;
ensures \result ==> balanceOf(\old(recipient)) == \old(balanceOf(recipient) +
amount)
                  && balanceOf(\old(sender)) == \old(balanceOf(sender) - amount);
  also
requires recipient == sender;
ensures \result ==> balanceOf(\old(recipient)) == \old(balanceOf(recipient));
```

erc20-transferfrom-fail-exceed-allowance

Any call of the form `transferFrom(from, dest, amount)` with a value for `amount` that exceeds the allowance of address `msg.sender` must fail.

Specification:

```
requires msg.sender != sender;
requires amount > allowance(sender, msg.sender);
ensures !\result;
```

erc20-transferfrom-fail-exceed-balance

Any call of the form `transferFrom(from, dest, amount)` with a value for `amount` that exceeds the balance of address `from` must fail.

Specification:

```
requires amount > balanceOf(sender);
ensures !\result;
```

erc20-transferfrom-fail-recipient-overflow

Any call of `transferFrom(from, dest, amount)` with a value in `amount` whose transfer would cause an overflow of the balance of address `dest` must fail.

Specification:

```
requires recipient != sender;
requires balanceOf(recipient) + amount > type(uint256).max;
ensures !\result;
```

erc20-transferfrom-false

If `transferFrom` returns `false` to signal a failure, it must undo all incurred state changes before returning to the caller.

Specification:

```
ensures !\result ==> \assigned (\nothing);
```

erc20-transferfrom-never-return-false

The `transferFrom` function must never return `false` .

Specification:

```
ensures \result;
```

erc20-transferfrom-revert-zero-argument

All calls of the form `transferFrom(from, dest, amount)` must fail for transfers from or to the zero address.

Specification:

```
ensures \old(sender) == address(0) ==> !\result;
also
ensures \old(recipient) == address(0) ==> !\result;
```

Properties related to function `balanceOf`

erc20-balanceof-change-state

Function `balanceOf` must not change any of the contract's state variables.

Specification:

```
assignable \nothing;
```

erc20-balanceof-correct-value

Invocations of `balanceOf(owner)` must return the value that is held in the contract's balance mapping for address `owner`.

Specification:

```
ensures \result == balanceOf(\old(account));
```

erc20-balanceof-succeed-always

Function `balanceOf` must always succeed if it does not run out of gas.

Specification:

```
reverts_only_when false;
```

Properties related to function `allowance`

erc20-allowance-change-state

Function `allowance` must not change any of the contract's state variables.

Specification:

```
assignable \nothing;
```

erc20-allowance-correct-value

Invocations of `allowance(owner, spender)` must return the allowance that address `spender` has over tokens held by address `owner`.

Specification:

```
ensures \result == allowance(\old(owner), \old(spender));
```

erc20-allowance-succeed-always

Function `allowance` must always succeed, assuming that its execution does not run out of gas.

Specification:

```
reverts_only_when false;
```

Properties related to function `totalSupply`

erc20-totalsupply-change-state

The `totalSupply` function in contract FundController must not change any state variables.

Specification:

```
assignable \nothing;
```

erc20-totalsupply-correct-value

The `totalSupply` function must return the value that is held in the corresponding state variable of contract FundController.

Specification:

```
ensures \result == totalSupply();
```

erc20-totalsupply-succeed-always

The function `totalSupply` must always succeeds, assuming that its execution does not run out of gas.

Specification:

```
reverts_only_when false;
```

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.